

10 — Script de démonstration DEV (10–15 min)

Ton objectif : montrer une plateforme **fonctionnelle, déployée en ligne, multi-rôles**. Rester fluide, orienté produit. Avoir 2 onglets ouverts : **prod** `https://upcycleconnect.pro` (preuve non-localhost) et **local** `http://localhost:8088` (pour la modif en direct).

Avant de commencer (30 s)

- « L'application s'appelle **UpcycleConnect**, une plateforme d'économie circulaire : les particuliers donnent ou vendent des objets, les professionnels récupèrent la matière, le tout via des **conteneurs connectés**. »
- « Elle est **déployée en ligne** sur `https://upcycleconnect.pro`, en Docker derrière Nginx et HTTPS. »

Déroulé

1. Page d'accueil + connexion (1 min)

- Ouvrir `https://upcycleconnect.pro/` → landing.
- Montrer le **sélecteur de langue** (multilingue) et l'URL propre (`/login`, `/annonces` ...).
- Se connecter en **Utilisateur** (`test.client@renova.test`).

2. Espace particulier (2 min) — compte Utilisateur

- Tableau de bord : **stats dynamiques** (Annonces, Dépôt actif, **Score éco**).
- Créer une **annonce** (titre, catégorie, prix/don, photo) → « soumise à validation ».
- Aller sur `/annonces` : filtre **catégories depuis la base**, recherche, tri.

3. Espace professionnel (2 min) — compte Pro

- Se connecter Pro (`test.pro@renova.test`) → `/pro` (interface **teal**).
- Montrer abonnement Premium, projets **avant/après** avec impact CO₂.
- Ouvrir `/annonces` : mêmes annonces mais **aux couleurs pro** (thème adaptatif).

4. Back-office admin (3 min) — compte Admin ★ point fort

- Se connecter Admin (`test.admin@renova.test`) → `/admin`.
- **Vue d'ensemble** : KPI (utilisateurs, revenus, conteneurs), **activité récente dynamique**, badge de validations.
- **Validations** : approuver l'annonce créée à l'étape 2 → elle disparaît **en direct**.
- **Utilisateurs** (`/admin/users`) : **pagination**, validation/refus d'un compte.
- **Conteneurs** (`/admin/conteneurs`) : cliquer un conteneur → casiers, puis « **Rapport logistique** » → **téléchargement d'un PDF** généré.
- **Documents** : factures/contrats PDF.

5. Preuve technique : déploiement + API (2 min)

- Montrer un terminal : `docker ps` (3 conteneurs Up), `docker compose logs backend`.
- Montrer un appel API en direct (Postman/curl) : `GET /admin/users` avec le token.
- « L'API Go tourne sur `:8081`, protégée par **JWT** et vérification de **rôle côté serveur**. »

6. Modification en direct (si demandée, 2–3 min)

- Prendre un exercice simple préparé (voir `12_EXERCICES`), ex : changer le **taux de commission** dans `API/admin/stripe.go` , ou ajouter une **colonne** dans le tableau admin.
- `docker compose up -d --build backend` (ou front) → Ctrl+Shift+R → montrer le résultat.

À dire à l'oral (phrases prêtes)

- « Front et back sont **séparés** et déployés en deux conteneurs distincts. »
- « Toute action sensible est **vérifiée côté serveur** : le front n'est jamais une source de confiance. »
- « Les fichiers uploadés sont stockés dans un **volume Docker persistant**, jamais en base. »

À ÉVITER de montrer

- Le flux de **paiement Stripe complet** (dépend de comptes Connect + webhook — risque de blocage live). Dire « le paiement passe par Stripe Checkout + Connect, avec commission de 5 % ».
- Le **push OneSignal** en local (désactivé sur localhost, HTTPS requis).
- Les pages/tables de données incohérentes ou de test brut (comptes « Rejeté », données `azerty...`).
- Ouvrir la console avec des erreurs CORS **en local** (n'existent pas en prod) : démontrer plutôt sur la prod.